

Created on....17.08.2016 Last changed on/by..... 29.06.2026 ABAP7
Author.....ABAP4

Title.....Interface between SAP ASRS Report

Type.....1 Reporting
Status.....
Application.....
Class.....
Logical database.....
 from Application....
Selection screen.....
Authorization group...
Package.....ZMM

 Start by Variants
 Deactivate editor lock
X Fixed point arithmetic active

```
1  *&-----*
2  *& Report  ZMM_ASRS_SAP_INTERFACE
3  *&
4  *&-----*
5  *&
6  *&
7  *&-----*
8
9  REPORT zmm_asrs_sap_interface.
10
11
12  TABLES: zmm_asrs.
13
14  TYPES: BEGIN OF ty_asrs,
15          msg_src TYPE zmsg_src,
16          msg_rec_id TYPE zmsg_rec_id,
17          msg_trans_type TYPE zmsg_trans_type,
18          msg_action TYPE zmsg_action,
19          msg_ret_src TYPE zmsg_ret_src,
20          msg_ret_rec_id TYPE zmsg_ret_rec_id,
21          msg_ret_trans_id TYPE zmsg_ret_trans_id,
22          msg_dt_def TYPE zdates,
23          gr_no TYPE mblnr,
24          req_id TYPE zreq_id,
25          req_type TYPE zreq_type,
26          mat_code TYPE matnr,
27          sap_batch TYPE charg_d,
28          qty TYPE erfmq,
29          status TYPE zstat,
30          old_status TYPE zold_status,
31          total_pack TYPE qanzgeb,
32          mfg_date TYPE zmfg_date,
33          manufacturer TYPE zmanufacturer,
34          mfg_batch TYPE zmfg_batch,
35          line_item TYPE mblpo,
36          plant TYPE werks_d,
37          sent TYPE icon_d,
38          rec TYPE icon_d,
39          proc TYPE icon_d,
40  END OF ty_asrs.
41
42  *SELECT-OPTIONS: s_werks FOR zmm_asrs-plant.
43  *PARAMETERS p_date TYPE datum OBLIGATORY.
44
45  PARAMETERS s_werks TYPE zmm_asrs-plant OBLIGATORY.
46  SELECT-OPTIONS: p_date FOR sy-datum OBLIGATORY.
47
48
49  DATA: gt_asrs TYPE STANDARD TABLE OF zmm_asrs,
50          gt_con TYPE STANDARD TABLE OF zmm_dbcon_asrs,
51          gs_con TYPE zmm_dbcon_asrs,
52          gt_final TYPE STANDARD TABLE OF ty_asrs.
53  *
54  *AT SELECTION-SCREEN ON p_date.
55  *  IF p_date-high IS INITIAL.
56  *    MESSAGE 'Fill all required fields' TYPE 'E'.
57  *  ENDIF.
58
59
60  START-OF-SELECTION.
61    PERFORM get_data.
```

```

62     PERFORM process_data.
63
64 END-OF-SELECTION.
65     PERFORM display_alv.
66 *&-----*
67 *&      Form   GET_DATA
68 *&-----*
69 *      text
70 *-----*
71 *  -->  p1      text
72 *  <--  p2      text
73 *-----*
74 FORM get_data .
75     SELECT msg_src
76            msg_rec_id
77            msg_trans_type
78            msg_action
79            msg_dt_def
80            msg_stat
81            gr_no
82            req_id
83            req_type
84            mat_code
85            sap_batch
86            qty
87            status
88            old_status
89            total_pack
90            mfg_date
91            manufacturer
92            mfg_batch
93            line_item
94            plant
95            trf_status
96     FROM zmm_asrs
97     INTO CORRESPONDING FIELDS OF TABLE gt_asrs
98     WHERE msg_dt_def IN p_date
99     AND plant = s_werks.
100
101     SELECT *
102     FROM zmm_dbcon_asrs
103     INTO TABLE gt_con
104     WHERE werks = s_werks.
105
106 ENDFORM.                                " GET_DATA
107 *&-----*
108 *&      Form   PROCESS_DATA
109 *&-----*
110 *      text
111 *-----*
112 *  -->  p1      text
113 *  <--  p2      text
114 *-----*
115 FORM process_data .
116     DATA: ls_asrs TYPE zmm_asrs,
117            con(20) TYPE c,
118            ls_final TYPE ty_asrs.
119     DATA: exc_ref   TYPE REF TO cx_sy_native_sql_error,
120            error_text TYPE string.
121     SORT gt_asrs BY plant.
122

```

```
123
124 LOOP AT gt_asrs INTO ls_asrs.
125     MOVE-CORRESPONDING ls_asrs TO ls_final.
126     ON CHANGE OF ls_asrs-plant.
127         READ TABLE gt_con INTO gs_con WITH KEY werks = ls_asrs-plant.
128         IF sy-subrc <> 0.
129             CONTINUE.
130         ENDIF.
131         TRY.
132             IF con IS NOT INITIAL.
133                 EXEC SQL.
134                     DISCONNECT :con
135                 ENDEXEC.
136             ENDIF.
137             CLEAR con.
138             EXEC SQL. "#EC CI_EXECSQL
139                 CONNECT TO :gs_con-DBCON
140             ENDEXEC.
141             EXEC SQL. "#EC CI_EXECSQL
142                 SET CONNECTION :gs_con-DBCON
143             ENDEXEC.
144             CATCH cx_sy_native_sql_error INTO exc_ref.
145         ENDTRY.
146
147     ENDON.
148     TRY.
149         EXEC SQL.
150             OPEN dbcur FOR
151             SELECT MSG_ERR, MSG_STAT
152             FROM HOST_TO_WMS
153             WHERE MSG_REC_ID = :ls_asrs-MSG_REC_ID
154         ENDEXEC.
155         EXEC SQL. "#EC CI_EXECSQL
156             FETCH NEXT dbcur INTO :ls_asrs-MSG_ERR, :ls_asrs-MSG_STAT
157         ENDEXEC.
158         EXEC SQL. "#EC CI_EXECSQL
159             close dbcur
160         ENDEXEC.
161         CATCH cx_sy_native_sql_error INTO exc_ref.
162
163     ENDTRY.
164     IF sy-subrc <> 0.
165         ls_final-rec = '@F1@'.
166         ls_final-proc = '@KB@'.
167     ELSE.
168         ls_final-rec = '@DF@'.
169         IF ls_asrs-msg_stat = 'TRM'.
170             ls_final-proc = '@C9@'.
171         ELSEIF ls_asrs-msg_stat = 'DEF'.
172             ls_final-proc = '@7C@'.
173         ELSE.
174             ls_final-proc = '@AG@'.
175         ENDIF.
176     ENDIF.
177
178     IF ls_asrs-trf_status = 'Y'.
179         ls_final-sent = '@B4@'.
180     ELSE.
181         ls_final-sent = '@B2@'.
182     ENDIF.
183
```

```

184     APPEND ls_final TO gt_final.
185     CLEAR: ls_final, ls_asrs.
186 ENDLOOP.
187
188 IF con IS NOT INITIAL.
189     EXEC SQL.
190         DISCONNECT :con
191     ENDEXEC.
192 ENDIF.
193
194 ENDFORM.                                " PROCESS_DATA
195 *&-----*
196 *&      Form  DISPLAY_ALV
197 *&-----*
198 *      text
199 *-----*
200 *  —>  p1      text
201 *  <—  p2      text
202 *-----*
203
204 FORM display_alv .
205
206     DATA: go_alv TYPE REF TO cl_salv_table,
207            go_columns TYPE REF TO cl_salv_columns,
208            go_column TYPE REF TO cl_salv_column,
209            go_func TYPE REF TO cl_salv_functions,
210            go_layout TYPE REF TO cl_salv_layout.
211
212     DATA: lv_small TYPE scrtext_s,
213            lv_large TYPE scrtext_l,
214            lv_med TYPE scrtext_m.
215
216     DATA : ls_ref TYPE salv_s_ddic_reference.
217     DATA : lo_filters TYPE REF TO cl_salv_filters.
218     DATA : lv_edit TYPE lvc_edtmsk.
219     TRY.
220         CALL METHOD cl_salv_table=>factory
221     *             EXPORTING
222     *                 list_display  = IF_SALV_C_BOOL_SAP=>FALSE
223     *                 r_container   =
224     *                 container_name =
225         IMPORTING
226             r_salv_table = go_alv
227         CHANGING
228             t_table      = gt_final.
229
230     CALL METHOD go_alv->get_columns
231     RECEIVING
232         value = go_columns.
233     TRY.
234         CALL METHOD go_columns->get_column
235         EXPORTING
236             columnname = 'SENT'
237         RECEIVING
238             value      = go_column.
239         lv_large = lv_med = lv_small = 'Sent to ASRS'.
240         go_column->set_long_text( lv_large ).
241         go_column->set_medium_text( lv_med ).
242         go_column->set_short_text( lv_small ).
243
244         CALL METHOD go_columns->get_column

```

```
245      EXPORTING
246          columnname = 'REC'
247      RECEIVING
248          value      = go_column.
249      lv_large = lv_med = lv_small = 'Recieved to ASRS'.
250      go_column->set_long_text( lv_large ).
251      go_column->set_medium_text( lv_med ).
252      go_column->set_short_text( lv_small ).
253
254      CALL METHOD go_columns->get_column
255          EXPORTING
256              columnname = 'PROC'
257          RECEIVING
258              value      = go_column.
259      lv_large = lv_med = lv_small = 'Processed by ASRS'.
260      go_column->set_long_text( lv_large ).
261      go_column->set_medium_text( lv_med ).
262      go_column->set_short_text( lv_small ).
263
264      **SOA by Vrushali on 17-03-2017 for ticket : 8000003027
265      CALL METHOD go_columns->get_column
266          EXPORTING
267              columnname = 'MAT_CODE'
268          RECEIVING
269              value      = go_column.
270      CLEAR ls_ref.
271      ls_ref-field = 'MAT_CODE'.
272      ls_ref-table = 'ZMM_ASRS'.
273      go_column->set_ddic_reference( ls_ref ).
274      go_column->set_edit_mask( ' ' ).
275      **EOA by Vrushali on 17.03.2017
276      CATCH cx_salv_not_found .
277      ENDTRY.
278
279      CALL METHOD go_alv->get_functions
280          RECEIVING
281              value = go_func.
282      go_func->set_all( abap_true ).
283      go_alv->display( ).
284      CATCH cx_salv_msg .
285      ENDTRY.
286
287
288      ENDFORM.                                " DISPLAY_ALV
```

Title:
Interface between SAP ASRS Report

Selection texts:

P_DATE
Run Date
S_WERKS
D .

Subroutines:		Line	Page
DISPLAY_ALV.....	ZMM_ASRS_SAP_INTERFACE	65	3
	ZMM_ASRS_SAP_INTERFACE	204	5
GET_DATA.....	ZMM_ASRS_SAP_INTERFACE	61	2
	ZMM_ASRS_SAP_INTERFACE	74	3
PROCESS_DATA.....	ZMM_ASRS_SAP_INTERFACE	62	3
	ZMM_ASRS_SAP_INTERFACE	115	3

DATABASE OPERATIONS

SELECT:

		Line	Page
ZMM_ASRS.....	ZMM_ASRS_SAP_INTERFACE	96	3
ZMM_DBCON_ASRS.....	ZMM_ASRS_SAP_INTERFACE	102	3

INTERNAL TABLE OPERATIONS

READ TABLE:

	Line	Page
GT_CON..... ZMM_ASRS_SAP_INTERFACE	127	4
